

Indian Institute of Technology Jodhpur

Problem Number 4: Cryptography on GPU

**Group 29**

# **Project Report**

## Parallel SHA-256 on CUDA: GPU Programming and Performance

A data-parallel batch hashing case study

### **Submitted by:**

Anand Pal(G25AIT1019), g25ait1019@iitj.ac.in

Arundhati(G25AIT1033), g25ait1033@iitj.ac.in

Karan Kapoor(G25AIT1233), g25ait1233@iitj.ac.in

Mohshinsha Harunsha Shahmadar(G25AIT1093), g25ait1093@iitj.ac.in

Mudrik Kaushik(G25AIT1096), g25ait1096@iitj.ac.in

June 30, 2026

# Contents

|          |                                              |          |
|----------|----------------------------------------------|----------|
| <b>1</b> | <b>Abstract</b>                              | <b>3</b> |
| 1.1      | Problem Overview . . . . .                   | 3        |
| 1.2      | Objectives . . . . .                         | 3        |
| 1.3      | Proposed GPU-Based Solution . . . . .        | 3        |
| 1.4      | Key Results . . . . .                        | 3        |
| <b>2</b> | <b>Introduction</b>                          | <b>3</b> |
| 2.1      | Assignment Context . . . . .                 | 3        |
| 2.2      | Why This Workload Fits a GPU . . . . .       | 3        |
| 2.3      | SHA-256 as Application (Brief) . . . . .     | 4        |
| 2.4      | Objectives and Scope . . . . .               | 4        |
| <b>3</b> | <b>Background: GPU Programming Concepts</b>  | <b>4</b> |
| 3.1      | CPU vs GPU Execution Model . . . . .         | 4        |
| 3.2      | CUDA Thread Hierarchy . . . . .              | 4        |
| 3.3      | Memory Hierarchy (How We Use It) . . . . .   | 4        |
| 3.4      | Data Parallelism and Independence . . . . .  | 4        |
| <b>4</b> | <b>Parallelization Strategy</b>              | <b>5</b> |
| 4.1      | Why Parallelize This Way? . . . . .          | 5        |
| 4.2      | What Each GPU Thread Computes . . . . .      | 5        |
| 4.3      | Workload Division . . . . .                  | 5        |
| 4.4      | Launch Configuration . . . . .               | 5        |
| 4.5      | Input Data Distribution . . . . .            | 6        |
| 4.6      | Output Collection . . . . .                  | 6        |
| 4.7      | Dependencies Between Threads . . . . .       | 6        |
| 4.8      | Avoiding Race Conditions . . . . .           | 6        |
| <b>5</b> | <b>CUDA Implementation</b>                   | <b>6</b> |
| 5.1      | Overall Architecture . . . . .               | 6        |
| 5.2      | Data Flow . . . . .                          | 7        |
| 5.3      | Kernel Design . . . . .                      | 7        |
| 5.4      | Memory Allocation . . . . .                  | 7        |
| 5.5      | Memory Transfers (Host ↔ Device) . . . . .   | 7        |
| 5.6      | Kernel Launch and Synchronization . . . . .  | 8        |
| 5.7      | Important Implementation Decisions . . . . . | 8        |
| <b>6</b> | <b>Experimental Setup and Results</b>        | <b>8</b> |
| 6.1      | Hardware and Software . . . . .              | 8        |
| 6.2      | Benchmark Harness and Methodology . . . . .  | 8        |
| 6.3      | Correctness . . . . .                        | 9        |
| 6.4      | Real Dataset: 1M Messages . . . . .          | 9        |

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| 6.5      | Scaling Sweep (Synthetic)                       | 9         |
| 6.6      | Block-Size Sweep ( $N=1,000,000$ , Kernel-Only) | 9         |
| <b>7</b> | <b>Challenges and Limitations</b>               | <b>10</b> |
| 7.1      | Memory Transfer Overhead                        | 10        |
| 7.2      | Warp Divergence                                 | 10        |
| 7.3      | Occupancy and Register Pressure                 | 10        |
| 7.4      | Scalability                                     | 11        |
| 7.5      | Implementation Limits                           | 11        |
| <b>8</b> | <b>Conclusion</b>                               | <b>11</b> |
| 8.1      | Summary                                         | 11        |
| 8.2      | Key GPU Programming Findings                    | 11        |
| 8.3      | Benefits Demonstrated                           | 11        |

# 1 Abstract

## 1.1 Problem Overview

This assignment uses batch SHA-256 hashing as a vehicle to demonstrate GPU programming: mapping an embarrassingly parallel workload onto CUDA threads, managing device memory, launching kernels, and measuring speedup against a CPU baseline. SHA-256 provides a realistic application; the primary learning outcome is understanding *how* and *why* we parallelize on a GPU.

## 1.2 Objectives

- Design a data-parallel mapping from messages to GPU threads with no inter-thread dependencies.
- Implement host–device data movement, kernel launch, and result collection through a reusable host API.
- Validate correctness before benchmarking; analyze where time is spent (compute vs. PCIe transfer).

## 1.3 Proposed GPU-Based Solution

One CUDA thread hashes exactly one message. Inputs are packed in a single device buffer with per-message offset/length index arrays; outputs are written to disjoint 32-byte slots. A four-stage host pipeline generates data, invokes the GPU engine, validates digests, and benchmarks throughput.

## 1.4 Key Results

On an RTX 4060 Laptop GPU, one million messages achieve  $\sim 18\times$  end-to-end and  $\sim 63\times$  kernel-only speedup over a serial CPU baseline, with zero digest mismatches against both the CPU reference and `expected_digests.bin`. Roughly 70% of end-to-end time is host–device transfer—illustrating that GPU programming requires optimizing the full path, not just the kernel. Numbers are from the committed run in `results/benchmark_run.txt` (harness: `results/scale_benchmark.cu`).

# 2 Introduction

## 2.1 Assignment Context

Problem Number 4 asks us to accelerate a cryptographic primitive on the GPU. We chose SHA-256 batch hashing because it exposes the core GPU programming patterns—thread indexing, global memory access, constant memory, kernel launch configuration, and host–device synchronization—without requiring threads to communicate with one another [2].

## 2.2 Why This Workload Fits a GPU

Hashing  $N$  independent messages is a *map* operation: the same computation applied to  $N$  different inputs. There is no reduction, no shared partial state, and no read-after-write dependency between

messages. Such workloads achieve near-linear scaling with thread count until memory bandwidth or PCIe limits are hit.

## 2.3 SHA-256 as Application (Brief)

SHA-256 [1] compresses 512-bit blocks into a 256-bit digest. Each thread runs the full hash (padding + compression rounds) locally in registers and local stack memory. We do not parallelize *within* a single hash; we parallelize *across* many hashes.

## 2.4 Objectives and Scope

This report emphasizes parallelization strategy and CUDA implementation decisions. Scope: single GPU, CUDA (not OpenCL), batch mode, serial CPU reference via OpenSSL [3]. Out of scope: multi-GPU, within-message parallelism, and streaming/persistent kernels.

# 3 Background: GPU Programming Concepts

## 3.1 CPU vs GPU Execution Model

CPUs target low-latency execution on a few complex cores with large caches. GPUs target throughput: thousands of lightweight threads grouped into warps of 32 that execute the same instruction in SIMT fashion. Our kernel launches one logical thread per message; warps may diverge when message lengths differ.

## 3.2 CUDA Thread Hierarchy

Threads are indexed as `threadIdx.x + blockIdx.x * blockDim.x`. A *grid* of *blocks*, each with *threads*, covers all messages. Idle threads in the last block are masked by `if (i < num_messages)`.

## 3.3 Memory Hierarchy (How We Use It)

- **Global memory** — packed `messages` buffer, `offsets`, `lengths`, and `digests` output array on the device.
- **Constant memory** — SHA-256 round constants  $K[0..63]$  in `__constant__` space; broadcast efficiently to all threads.
- **Registers / local memory** — per-thread hash state, padding buffer (up to 128 bytes), and round workspace inside `sha256_hash`.
- **Shared memory** — not used; no inter-thread data sharing needed.

## 3.4 Data Parallelism and Independence

In our design, parallelism is *across messages*, not within the compression function. This is the correct granularity because SHA-256 rounds are inherently sequential within one message. Parallelizing across messages avoids synchronization entirely.

## 4 Parallelization Strategy

This section answers how we decompose the workload for the GPU.

### 4.1 Why Parallelize This Way?

**Alternative considered:** parallelize rounds or blocks *within* one message—rejected because each compression round depends on the previous state; threads would need frequent barriers and shared memory, adding complexity with little gain for short messages.

**Chosen strategy: one thread per message** (embarrassingly parallel “map”). Each thread is self-contained: read bytes, pad, compress all blocks, write digest. This matches the assignment I/O contract and eliminates race conditions by construction.

### 4.2 What Each GPU Thread Computes

Thread with global index  $i$  (when  $i < N$ ):

1. Reads `offset = offsets[i]` and `len = lengths[i]`.
2. Calls `sha256_hash(messages + offset, len, digests + i*32)`.
3. Inside `sha256_hash`: processes all 512-bit blocks of message  $i$ , applies padding, runs 64-round compression per block, emits 32-byte digest.

No thread reads or writes another thread’s message or digest.

### 4.3 Workload Division

- Total work items:  $N = \text{num\_messages}$ .
- Thread  $i$  owns work item  $i$  exclusively.
- Remainder threads ( $i \geq N$ ) exit immediately—handles non-multiple grid sizes without extra logic on the host.

### 4.4 Launch Configuration

Table 1: Kernel launch parameters (fixed in our implementation).

| Parameter                                     | Value                                   |
|-----------------------------------------------|-----------------------------------------|
| Threads per block ( <code>blockDim.x</code> ) | 256                                     |
| Number of blocks ( <code>gridDim.x</code> )   | $\lceil N/256 \rceil$                   |
| Total threads launched                        | $256 \times \lceil N/256 \rceil \geq N$ |
| Active threads                                | $N$                                     |

**Why 256 threads/block?** It is a common default that balances occupancy and register pressure on NVIDIA hardware. Our block-size sweep (Section 6) shows kernel time varies only  $\sim 7\%$  across 64–1024 threads/block, so the default of 256 is a safe, contract-compliant choice.

## 4.5 Input Data Distribution

All messages are stored *contiguously* in one host/device byte array (`messages.bin`). Two integer arrays describe boundaries:

- `offsets[i]` — byte index where message  $i$  starts.
- `lengths[i]` — byte length of message  $i$ .

The host copies the entire packed buffer plus both index arrays to device global memory in three `cudaMemcpy` H2D calls. Threads index into the *same* `d_messages` pointer at different offsets—no per-thread scatter/gather setup on the device.

## 4.6 Output Collection

Thread  $i$  writes its 32-byte digest to `d_digests[i*32 .. i*32+31]`. Output regions are disjoint and fixed size, so writes never overlap. After `cudaDeviceSynchronize()`, the host copies the full  $N \times 32$  byte digest buffer back with one D2H `cudaMemcpy`.

## 4.7 Dependencies Between Threads

**None.** Messages are independent; digests are independent. No thread reads a digest produced by another thread. No `__syncthreads()` is required inside the kernel.

## 4.8 Avoiding Race Conditions

Race freedom follows from the mapping:

- **Writes:** thread  $i$  writes only `digests + i*32` (unique range).
- **Reads:** thread  $i$  reads only `messages + offsets[i]` for `lengths[i]` bytes (non-overlapping input ranges by dataset contract).
- **Shared constants:** `K[]` in constant memory is read-only for all threads.
- **No atomics needed:** disjoint output slots make atomic operations unnecessary.

# 5 CUDA Implementation

We describe architecture and data flow—not a line-by-line listing of source code.

## 5.1 Overall Architecture

The project splits into host tools and a linkable GPU *engine*:

- `sha256_gpu.cu` — implements `sha256_gpu_hash()`: allocation, copies, launch, result return. No `main()`; linked by validator and benchmark.
- `sha256.cuh` — device functions and `sha256_kernel` (the `__global__` entry point).
- Host drivers — `cpu_reference` (dataset), `hash_dataset` (batch driver), `validate`, `benchmark`.

Shared types and API live in `include/sha256_gpu.hpp` and `IO_CONTRACT.md` so all modules agree on data layout.

## 5.2 Data Flow

Figure 1 shows the end-to-end path from dataset generation to timed benchmark.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Host (CPU)</b><br><code>cpu_reference</code> $\rightarrow$ <code>data/</code> files (messages, offsets, lengths, expected digests)<br><b>Host <math>\rightarrow</math> Device</b><br><code>sha256_gpu_hash()</code> : <code>cudaMalloc</code> + H2D copy<br><b>Device (GPU)</b><br><code>sha256_kernel</code> : one thread per message $\rightarrow$ <code>d_digests</code><br><b>Device <math>\rightarrow</math> Host</b><br>D2H copy $\rightarrow$ <code>std::vector</code> of $N \times 32$ bytes<br><b>Host (CPU)</b><br><code>validate</code> / <code>benchmark</code> compare or time results |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Figure 1: Host–device data flow for batch SHA-256.

## 5.3 Kernel Design

`sha256_kernel` is a thin `__global__` wrapper:

```
i = threadIdx.x + blockIdx.x * blockDim.x;
if (i < num_messages)
    sha256_hash(messages + offsets[i], lengths[i],
                digests + i * 32);
```

Heavy work lives in device functions (`sha256_transform`, `sha256_hash`) marked `__device__ inline`. The kernel itself contains no loops over messages—only over compression rounds inside each thread’s private execution.

## 5.4 Memory Allocation

For  $N$  messages and `total_bytes` packed payload, the engine allocates: All buffers are freed in a

Table 2: Device buffers allocated per `sha256_gpu_hash()` call.

| Pointer                 | Size                     | Purpose                 |
|-------------------------|--------------------------|-------------------------|
| <code>d_messages</code> | <code>total_bytes</code> | Packed input bytes      |
| <code>d_offsets</code>  | $N \times 4$ bytes       | Start index per message |
| <code>d_lengths</code>  | $N \times 4$ bytes       | Length per message      |
| <code>d_digests</code>  | $N \times 32$            | Output digests          |

`cleanup()` lambda on both success and exception paths—`cudaFree(nullptr)` is safe if allocation partially failed.

## 5.5 Memory Transfers (Host $\leftrightarrow$ Device)

Host to device (before kernel):



- `cudaMemcpy(d_messages, messages, total_bytes, H2D)`
- `cudaMemcpy(d_offsets, offsets, N*sizeof(int), H2D)`
- `cudaMemcpy(d_lengths, lengths, N*sizeof(int), H2D)`

**Device to host (after kernel):**

- `cudaMemcpy(digests, d_digests, N*32, D2H)`

Benchmarking shows these copies dominate end-to-end time for short messages (Table 5)—a key GPU programming lesson: the PCIe bus is part of the performance model.

## 5.6 Kernel Launch and Synchronization

```
sha256_kernel<<<numBlocks, 256>>>(  
    d_messages, d_offsets, d_lengths, d_digests, N);  
cudaGetLastError();           // catch launch config errors  
cudaDeviceSynchronize(); // wait for all threads; catch runtime errors
```

`cudaDeviceSynchronize()` ensures digests are complete before D2H copy. The host API throws `std::runtime_error` on any CUDA failure so callers (`validate`, `benchmark`) can report errors without aborting the process.

## 5.7 Important Implementation Decisions

- **Packed buffer + index arrays** rather than an array of per-message device pointers—one H2D copy for payload, simpler allocator, better PCIe efficiency than  $N$  small transfers.
- **Constant memory for  $K[]$**  — round constants read by every thread; constant cache broadcasts efficiently.
- **No shared memory** — would only help if threads cooperated; they do not.
- **Guard  $i < \text{num\_messages}$**  — standard CUDA idiom for arbitrary  $N$ .
- **size\_t for digest offset ( $i * 32$ )** — avoids overflow past  $\sim 67\text{M}$  messages on 32-bit indexing.
- **Correctness before speed** — smoke test, edge-case validator, and full-dataset compare run before benchmark numbers are trusted.

# 6 Experimental Setup and Results

## 6.1 Hardware and Software

## 6.2 Benchmark Harness and Methodology

Scale benchmarks were run with `results/scale_benchmark.cu`, which splits GPU time using CUDA events (H2D, kernel, D2H), warms up the GPU for  $\sim 300$  ms before timing, repeats each point 10–50 times, and reports min/median/mean/stddev (we quote medians). Every point verifies GPU

Table 3: Benchmark environment (from `results/benchmark.md`).

| Item          | Value                                                  |
|---------------|--------------------------------------------------------|
| GPU           | NVIDIA GeForce RTX 4060 Laptop (24 SMs, 8.6 GB, sm_89) |
| CUDA          | Runtime 13.2, driver 13.2 (nvcc 13.0)                  |
| Host compiler | MSVC 14.44 (VS 2022 Build Tools)                       |
| OS            | Windows 11                                             |
| CPU baseline  | Single-threaded scalar SHA-256 (FIPS 180-4)            |

digests against the CPU reference; the 1M real-dataset run also checks `expected_digests.bin`. The canonical console log is `results/benchmark_run.txt`; full analysis is in `results/benchmark.md`.

Correctness during development uses OpenSSL via `validate.cpp`; the scale harness uses a self-contained scalar CPU SHA-256 verified against NIST vectors at startup.

### 6.3 Correctness

Table 4: Correctness gates from scale benchmark run (all passed).

| Check                                                     | Result        |
|-----------------------------------------------------------|---------------|
| Host SHA-256 vs. NIST vectors                             | Match         |
| GPU vs. CPU reference (1M real + synthetic sweep)         | 0 mismatches  |
| GPU vs. <code>expected_digests.bin</code> (1M real data/) | Byte-for-byte |

### 6.4 Real Dataset: 1M Messages

Table 5 breaks down end-to-end GPU time on the Karan-generated `data/` set. PCIe transfer dominates ( $\sim 70\%$ ); kernel compute is  $\sim 29\%$  of wall time.

Table 5: GPU time breakdown, 1M real messages (median).

| Stage        | ms           | Share       |
|--------------|--------------|-------------|
| H2D copy     | 3.59         | 31%         |
| Kernel       | 3.39         | 29%         |
| D2H copy     | 4.54         | 39%         |
| <b>Total</b> | <b>11.55</b> | <b>100%</b> |

### 6.5 Scaling Sweep (Synthetic)

### 6.6 Block-Size Sweep ( $N=1,000,000$ , Kernel-Only)

**GPU programming reading of results:**

- Below  $\sim 100\text{K}$  messages, launch + PCIe overhead caps end-to-end speedup at  $\sim 11\times$ ; above 100K the GPU saturates.
- Kernel-only speedup reaches  $\sim 55\text{--}63\times$  once saturated; end-to-end plateaus at  $\sim 16\text{--}18\times$  because transfer is  $\sim 70\%$  of wall time.

Table 6: CPU vs GPU throughput, 1M real messages (median).

| Metric           | CPU          | GPU                         | Speedup       |
|------------------|--------------|-----------------------------|---------------|
| Time (ms)        | 212.2        | 11.55 (e2e) / 3.39 (kernel) | 18.4× / 62.5× |
| Throughput       | 4.7 M hash/s | 86.6 M hash/s (e2e)         | 18.4×         |
| Kernel bandwidth | —            | 9.43 GB/s                   | —             |

Table 7: Scaling sweep (median, ~32-byte messages). Source: `results/benchmark_run.txt`.

| $N$  | Input   | CPU ms | GPU e2e | GPU kernel | e2e × | kernel × | OK |
|------|---------|--------|---------|------------|-------|----------|----|
| 10K  | 0.32 MB | 2.16   | 0.190   | 0.041      | 11.4× | 52.8×    | ✓  |
| 100K | 3.25 MB | 20.13  | 1.239   | 0.371      | 16.3× | 54.3×    | ✓  |
| 1M   | 32.5 MB | 202.30 | 11.78   | 3.438      | 17.2× | 58.8×    | ✓  |
| 10M  | 325 MB  | 2035.5 | 119.8   | 34.09      | 17.0× | 59.7×    | ✓  |

- Block-size tuning (64–1024) spans only a ~7% kernel-time band—the workload is memory- and PCIe-bound, not occupancy-limited. Default 256 is fine.
- CPU throughput stays flat at ~4.9 M hash/s (serial loop); GPU kernel throughput scales with  $N$  until bandwidth-saturated.

## 7 Challenges and Limitations

### 7.1 Memory Transfer Overhead

PCIe bandwidth caps end-to-end throughput. For small messages the payload plus index metadata is large relative to useful compute per thread.

### 7.2 Warp Divergence

Variable `lengths[i]` causes threads in the same warp to execute different numbers of compression rounds—some threads finish early and wait (SIMT divergence).

### 7.3 Occupancy and Register Pressure

Each thread holds hash state, a 64-word workspace, and up to 128 bytes of padding buffer in registers/local memory. This limits how many threads per SM can be active simultaneously, though our kernel remains memory-bound.

Table 8: Block-size sensitivity (median kernel time).

| Threads/block | Kernel (ms) | Hashes/s | GB/s |
|---------------|-------------|----------|------|
| 64            | 3.28        | 305 M    | 9.91 |
| 128           | 3.39        | 295 M    | 9.60 |
| 256           | 3.47        | 288 M    | 9.36 |
| 512           | 3.53        | 283 M    | 9.20 |
| 1024          | 3.49        | 287 M    | 9.31 |

## 7.4 Scalability

Single-GPU only. Multi-GPU would need dataset partitioning and a second-level merge with no digest dependencies between partitions.

## 7.5 Implementation Limits

Batch API reallocates device memory each call; no pinned memory, CUDA streams, or persistent device-resident datasets yet—natural next steps for a GPU programming iteration.

# 8 Conclusion

## 8.1 Summary

We parallelized batch SHA-256 using a one-thread-per-message CUDA mapping with disjoint inputs and outputs, eliminating synchronization and races. A host engine handles allocation, H2D/D2H copies, kernel launch, and error-safe cleanup.

## 8.2 Key GPU Programming Findings

- Choose parallelism granularity where tasks are independent—here, across messages, not within one hash.
- The full pipeline (copy + kernel + copy) determines user-visible speedup.
- Constant memory, launch bounds, and guards are practical tools that matter as much as the device algorithm.
- Correctness validation on the GPU path is mandatory before trusting timing.

## 8.3 Benefits Demonstrated

The GPU sustains  $\sim 87$  M hashes/s end-to-end (and  $\sim 300$  M+ kernel-only at tuned block sizes) on saturated workloads—suitable for bulk integrity checking. Measured speedups ( $\sim 18\times$  e2e,  $\sim 63\times$  kernel) confirm the parallel mapping is sound; further gains require overlapping transfers with compute (streams, pinned memory) rather than micro-optimizing SHA-256 rounds.

# References

- [1] National Institute of Standards and Technology. Secure hash standard (FIPS PUB 180-4). <https://csrc.nist.gov/publications/detail/fips/180/4/final>, 2015.
- [2] NVIDIA Corporation. CUDA C++ programming guide. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>.
- [3] OpenSSL Project. Openssl cryptography and SSL/TLS toolkit. <https://www.openssl.org/>.